

Lista de 20 questões - Padrões de Projeto e SOLID

Questão 1 - Responsabilidade Única

Explique o **Princípio da Responsabilidade Única** e analise o código abaixo.

```
class Pedido {  
    constructor(  
        public cliente: string,  
        public valor: number  
    ) {}  
  
    public calcularTotal(): number {  
        return this.valor * 1.1;  
    }  
  
    public salvarNoBanco(): void {  
        console.log("Salvando pedido no banco");  
    }  
  
    public enviarEmailConfirmacao(): void {  
        console.log(`Enviando e-mail para ${this.cliente}`);  
    }  
}
```

Responda:

1. Qual princípio está sendo violado?
2. Por que ele está sendo violado?
3. Como o código poderia ser reorganizado?

Questão 2 - Factory

Explique o padrão **Factory**.

Na sua resposta, comente:

1. Qual problema ele resolve.
2. Por que espalhar `new ClasseConcreta()` pelo código pode ser ruim.
3. Como uma Factory reduz o acoplamento.
4. Dê um exemplo simples usando notificações ou canais de comunicação.

Questão 3 - Inversão de Dependência

Analise o código abaixo:

```
class EmailService {
    public enviar(mensagem: string): void {
        console.log(`Enviando e-mail: ${mensagem}`);
    }
}

class PedidoService {
    private emailService = new EmailService();

    public finalizarPedido(): void {
        this.emailService.enviar("Pedido finalizado");
    }
}
```

Responda:

1. Qual princípio SOLID está sendo violado?
2. Por que esse código gera alto acoplamento?
3. Refaça o código respeitando o princípio.

Questão 4 - Observer

Desenhe o diagrama do padrão **Observer**.

O diagrama deve conter:

1. **Subject**;
2. **Observer**;
3. **ConcreteSubject**;
4. **ConcreteObserver**;
5. métodos como `addObserver()`, `removeObserver()`, `notifyObservers()` e `update()`.

Depois, explique o funcionamento do padrão usando o exemplo de uma classe **Temperatura** que notifica vários termômetros.

Questão 5 - Aberto/Fechado

Analise o código abaixo:

```
class CalculadoraDesconto {
    public calcular(tipoCliente: string, valor: number): number {
        if (tipoCliente === "comum") {
            return valor * 0.05;
        }

        if (tipoCliente === "vip") {
            return valor * 0.1;
        }
    }
}
```

```
    }  
  
    if (tipoCliente === "premium") {  
        return valor * 0.15;  
    }  
  
    return 0;  
}  
}
```

Responda:

1. Qual princípio está sendo violado?
2. Por que o código viola esse princípio?
3. Refaça o código respeitando o princípio.

Questão 6 - Singleton

Explique o padrão **Singleton**.

Na sua resposta, comente:

1. Qual problema ele tenta resolver.
2. Como ele normalmente é implementado.
3. Por que o construtor costuma ser privado.
4. Quais problemas de design o Singleton pode causar.

Questão 7 - Proxy

Explique o padrão **Proxy** usando o exemplo de uma busca de livros com cache.

Na sua resposta, comente:

1. Qual é o papel do Proxy.
2. Qual é o objeto real.
3. Como o Proxy pode adicionar cache sem alterar a classe original.
4. Cite outros três requisitos não-funcionais que poderiam ser implementados com Proxy.

Questão 8 - Adapter

Explique o padrão **Adapter** usando o exemplo de projetores multimídia.

Na sua resposta, identifique:

1. Quem é o **Target**.
2. Quem é o **Adapter**.
3. Quem é o **Adaptee**.
4. Qual princípio SOLID o Adapter ajuda a aplicar.

Questão 9 - Facade

Explique o padrão **Facade** usando o exemplo de um Home Theater.

Na sua resposta, comente:

1. Qual problema ele resolve.
 2. Como reduz o acoplamento do cliente.
 3. Por que ele melhora o Information Hiding.
 4. Qual princípio pode ser violado se a fachada fizer coisas demais.
-

Questão 10 - Decorator

Explique o padrão **Decorator** usando o exemplo da cafeteria.

Na sua resposta, comente:

1. Por que criar uma classe para cada combinação de bebida e adicional seria ruim.
 2. Como o Decorator usa composição.
 3. Como ele respeita o Princípio Aberto/Fechado.
 4. Dê um exemplo com **CafeExpresso**, **Leite** e **Chantilly**.
-

Questão 11 - Strategy

Analise o código abaixo:

```
class CalculadoraFrete {  
    public calcular(tipo: string, peso: number): number {  
        if (tipo === "sedex") {  
            return peso * 10;  
        }  
  
        if (tipo === "pac") {  
            return peso * 5;  
        }  
  
        if (tipo === "transportadora") {  
            return peso * 8;  
        }  
  
        return 0;  
    }  
}
```

Responda:

1. Qual padrão de projeto poderia melhorar esse código?

2. Qual princípio SOLID está sendo violado?
 3. Refaça o código usando o padrão escolhido.
-

Questão 12 - Template Method

Explique o padrão **Template Method** usando o exemplo de cálculo de salário.

Na sua resposta, comente:

1. O que é o método template.
 2. O que fica na classe abstrata.
 3. O que fica nas subclasses.
 4. Por que esse padrão se relaciona com Inversão de Controle.
-

Questão 13 - Visitor

Explique o padrão **Visitor** usando uma hierarquia de veículos.

Considere:

```
Veiculo
  Carro
  Onibus
  Motocicleta
```

Responda:

1. Qual problema o Visitor resolve.
 2. Para que serve o método `accept(visitor)`.
 3. Para que servem os métodos `visitCarro()`, `visitOnibus()` e `visitMotocicleta()`.
 4. Cite uma vantagem e uma desvantagem do Visitor.
-

Questão 14 - Inversão de Controle

Explique **Inversão de Controle**.

Na sua resposta, comente:

1. O que significa dizer que o controle foi invertido.
 2. Como isso aparece no Template Method.
 3. Como isso aparece em frameworks.
 4. Qual a diferença entre Inversão de Controle e Inversão de Dependência.
-

Questão 15 - Framework vs Biblioteca

Explique a diferença entre **framework** e **biblioteca**.

Na sua resposta, use a frase:

Biblioteca: você chama o código dela.
Framework: ele chama o seu código.

Depois, dê um exemplo de cada um.

Questão 16 - Design for Change

Explique o conceito de **Design for Change**.

Na sua resposta, comente:

1. Por que padrões de projeto ajudam em Design for Change.
 2. Qual é a relação com o Princípio Aberto/Fechado.
 3. Qual é a relação com baixo acoplamento.
 4. Dê um exemplo simples.
-

Questão 17 - Overengineering

Explique o conceito de **Overengineering**.

Na sua resposta, comente:

1. Quando o uso de padrões de projeto pode virar overengineering.
 2. Por que mais classes e interfaces nem sempre significam melhor projeto.
 3. Dê um exemplo de solução exagerada para um problema simples.
-

Questão 18 - Segregação de Interfaces

Analise o código abaixo:

```
interface Trabalhador {  
    trabalhar(): void;  
    comer(): void;  
    dormir(): void;  
}  
  
class Robo implements Trabalhador {  
    public trabalhar(): void {  
        console.log("Robô trabalhando");  
    }  
  
    public comer(): void {  
        throw new Error("Robô não come");  
    }  
}
```

```
public dormir(): void {  
    throw new Error("Robô não dorme");  
}
```

Responda:

1. Qual princípio SOLID está sendo violado?
2. Por que o código viola esse princípio?
3. Refaça o código corrigindo a violação.

Questão 19 - Liskov

Analise o código abaixo:

```
class Ave {  
    public voar(): void {  
        console.log("Voando");  
    }  
}  
  
class Pinguim extends Ave {  
    public voar(): void {  
        throw new Error("Pinguim não voa");  
    }  
}
```

Responda:

1. Qual princípio SOLID está sendo violado?
2. Por que **Pinguim** não substitui corretamente **Ave**?
3. Refaça o código corrigindo a violação.

Questão 20 - Questão integrada

Um sistema possui uma classe **RelatorioService** que:

- busca dados no banco;
- gera PDF;
- exporta CSV;
- envia e-mail;
- decide o formato com **if**;
- instancia diretamente **EmailService** e **MySQLRepository**.

Responda:

1. Quais princípios SOLID podem estar sendo violados?

2. Quais padrões de projeto poderiam ajudar?
3. Explique uma possível refatoração geral para melhorar esse projeto.